

231112048 - Jordanniel Saor Sihombing

Tugas / Latihan SHA256

1. Tujuan untuk membuktikan bahwa perubahan 1 bit input ke dalam SHA256 akan menghasilkan perbedaan output paling tidak 128 (50%). Buatlah programnya

Ini algoritmanya:

Input X dan Y (bedakan 1 bit antara X dan Y); panjang teks bebas

A = Hash (X) dan B = Hash (Y); A dan B panjang 256 bit (ubah ke biner)

For I = 1 to n /* n = 256 */

 Cek bit ke I untuk A dan B

 Jika beda maka jBeda=jBeda+1

Next I

Print jBeda

Contoh:

X= 'b' = 0110 0110

h(X) = e9d71f5ee7c92d6dc9e92ffdad17b8bd49418f98

Y= 'c' = 0110 0111

h(Y) = 84a516841ba77a5b4648de2cd0dfcb30ea46dbb4

Hasil h(X) dan h(Y) kita ubah ke 256 bit

Setelah itu lakukan pengecekan bit ke-1 sampai bit ke-256 dari h(x) dan h(y)

Teori mengatakan perbedaan antara h(x) dan h(y) setidaknya 128 bit (posisi)

Sifat ini dikenal dengan *avalanche effect*

Tunjukkan hasil eksekusinya dan pindahkan ke word bersama dengan source code anda

Anda boleh melakukan eksperimen sebanyak mungkin (Never Kill Your Curiosity/NKYC)

Web HTML sederhana SHA256 yang saya buat, bisa di coba : <https://jrdnniel.github.io/sha256Kripto>

Code py

```
1 import hashlib
2
3 def string_to_binary(text):
4     """Mengubah teks menjadi representasi biner (8-bit per karakter)"""
5     return ' '.join(format(ord(c), '08b') for c in text)
6
7 def sha256_hash_bin(text):
8     """Menghasilkan hash SHA-256 dalam bentuk Hex dan Biner (256-bit)"""
9     hex_hash = hashlib.sha256(text.encode('utf-8')).hexdigest()
10    bin_hash = bin(int(hex_hash, 16))[2:].zfill(256)
11    return hex_hash, bin_hash
12
13 def format_bin_spaced(bin_str, step=8):
14     """Memecah string biner panjang dengan spasi setiap 'step' karakter"""
15     return ' '.join(bin_str[i:i+step] for i in range(0, len(bin_str), step))
16
17 def generate_y_1bit_diff(text):
18     """Membuat teks Y yang berbeda tepat 1 bit dari teks X"""
19     if not text:
20         return text
21     char_list = list(text)
22     char_list[0] = chr(ord(char_list[0]) ^ 1)
23     return ''.join(char_list)
24
25 def print_difference_map(bin_A, bin_B):
26     """Mencetak peta perbandingan bit baris demi baris (64 bit per baris)"""
27     print("\n[ PETA PERBEDAAN BIT ]")
28     print("Keterangan: Tanda '^' di baris 'Beda' menunjukkan posisi bit yang berubah.")
29     chunk_size = 64
30
31     for i in range(0, 256, chunk_size):
32         chunk_A = bin_A[i:i+chunk_size]
33         chunk_B = bin_B[i:i+chunk_size]
34
35         spaced_A = format_bin_spaced(chunk_A)
36         spaced_B = format_bin_spaced(chunk_B)
37
38         # Membuat string indikator perbedaan ('^')
39         diff_str = ""
40         for a, b in zip(spaced_A, spaced_B):
41             if a == ' ':
42                 diff_str += ' ' # Pertahankan spasi pemisah
43             elif a != b:
44                 diff_str += '^' # Beri tanda jika bit berbeda
45             else:
46                 diff_str += ' ' # Kosongkan jika bit sama
47
48         print(f"\n>> Bit ke-{i+1:03d} sampai {i+chunk_size:03d}:")
49         print(f"h(X) : {spaced_A}")
50         print(f"h(Y) : {spaced_B}")
51         print(f"Beda : {diff_str}")
52
53 def execute_avalanche_test(x):
54     """Fungsi utama eksekusi"""
55     print("-" * 85)
56     y = generate_y_1bit_diff(x)
57
58     print(f"Input X = '{x}' -> Biner: {string_to_binary(x)}")
59     print(f"Input Y = '{y}' -> Biner: {string_to_binary(y)}")
60     print("-" * 85)
61
62     hex_A, bin_A = sha256_hash_bin(x)
63     hex_B, bin_B = sha256_hash_bin(y)
64
65     print(f"h(X) [Hex] = {hex_A}")
66     print(f"h(Y) [Hex] = {hex_B}")
67
68     # Panggil fungsi pencetak peta
69     print_difference_map(bin_A, bin_B)
70     print("-" * 85)
71
72     # Hitung total perbedaan
73     jBeda = sum(1 for i in range(256) if bin_A[i] != bin_B[i])
74     persentase = (jBeda / 256) * 100
75
76     print(f"Total bit yang berbeda {jBeda} = {jBeda} bit dari 256 bit")
77     print(f"Persentase Perubahan Output = {persentase:.2f}%")
78     print("-" * 85)
79
80 # =====
81 # MAIN PROGRAM (MODE INTERAKTIF)
82 # =====
83 if __name__ == "__main__":
84     print("PROGRAM PETA AVALANCHE EFFECT SHA-256")
85     while True:
86         user_input = input("\nMasukkan Input X (ketik 'exit' untuk keluar): ")
87         if user_input.lower() == 'exit':
88             break
89         if len(user_input) > 0:
90             execute_avalanche_test(user_input)
```

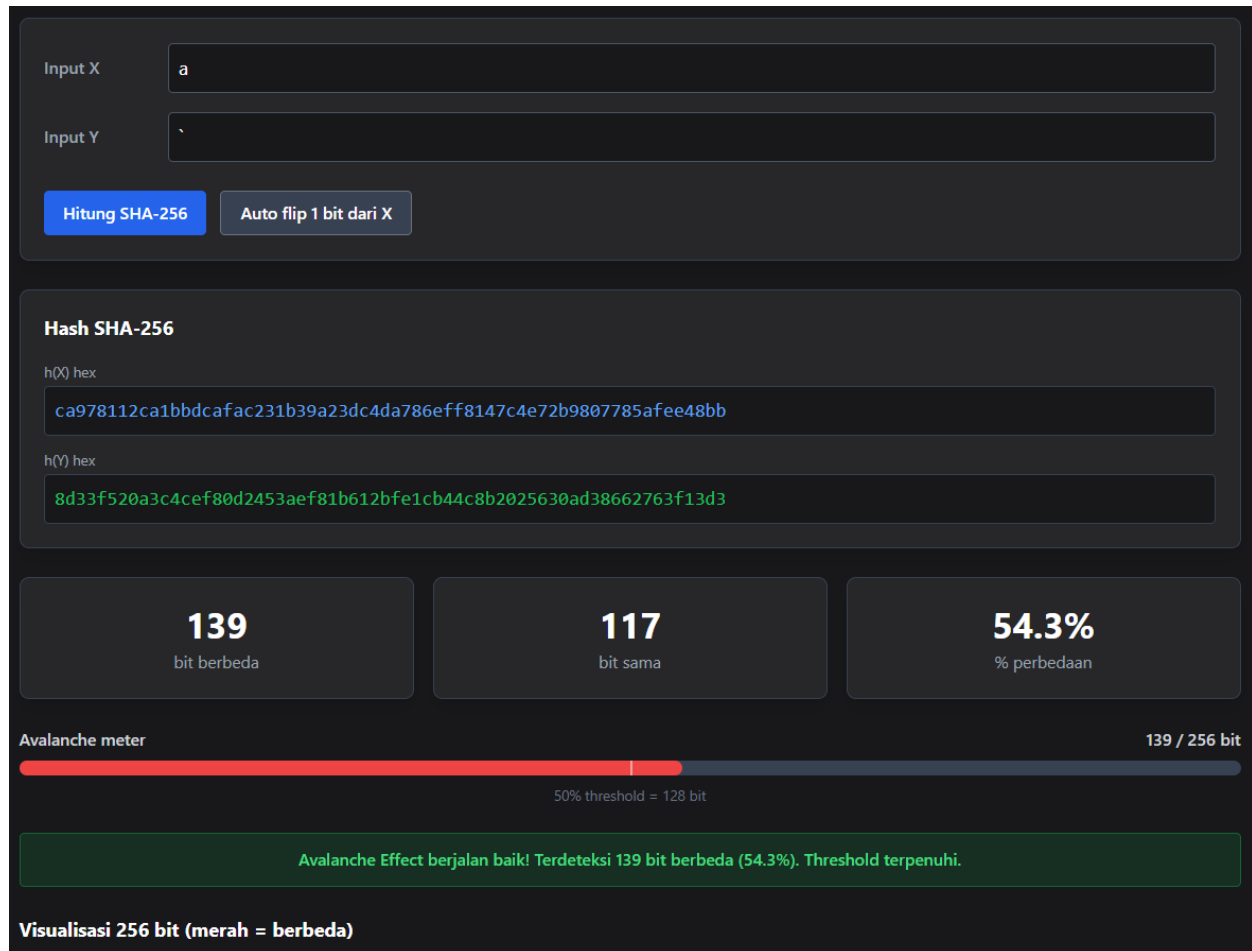
Percobaan 1:

Input X = 'a' -> Biner: 01100001

Input Y = '' -> Biner: 01100000

h(X) [Hex] = ca978112ca1bbdcfac231b39a23dc4da786eff8147c4e72b9807785afee48bb

h(Y) [Hex] = 8d33f520a3c4cef80d2453aef81b612bfe1cb44c8b2025630ad38662763f13d3



Penjelasan:

Input X = 'a' (biner: 01100001) dan Y = '' (biner: 01100000) hanya berbeda pada 1 bit terakhir (bit ke-0, LSB). Kedua string diproses oleh fungsi SHA-256 menggunakan hashlib Python. Algoritma SHA-256 melakukan:

- (1) padding pesan agar panjangnya kongruen $448 \bmod 512$ bit
- (2) menambahkan representasi 64-bit panjang pesan asli
- (3) inisialisasi delapan variabel hash H0-H7 dari konstanta bawaan
- (4) menjalankan 64 putaran fungsi kompresi dengan konstanta $K[0..63]$.

Hasil akhir adalah digest 256 bit dalam format heksadesimal: $h(X) = \text{ca978112...afee48bb}$ dan $h(Y) = \text{8d33f520...763f13d3}$. Kedua hash dikonversi ke biner 256 bit, lalu dibandingkan bit demi bit (loop $i=1$ sampai 256). Ditemukan 139 bit berbeda dan 117 bit sama, menghasilkan persentase perbedaan 54,3%. Karena 139 lebih besar dari threshold 128 (50%), Avalanche Effect dinyatakan **TERPENUHI**.

Visualisasi 256 bit



Penjelasan:

Setelah hash heksadesimal diperoleh, program mengkonversi nilai hex ke integer dengan `int(hex_hash, 16)`, lalu ke representasi biner 256 bit menggunakan `bin(...)[2:].zfill(256)`. Hasil biner ditampilkan dengan spasi setiap 8 bit sehingga terbentuk 32 kelompok yang merepresentasikan 32 byte. Pada bagian 'Peta perbedaan bit', setiap dari 256 posisi direpresentasikan sebagai kotak kecil dalam grid: kotak berwarna merah (berbeda) menandai posisi di mana bit $h(X)$ dan $h(Y)$ tidak sama, sedangkan kotak abu-abu (sama) menandai posisi yang identik.

Program mengiterasi setiap pasang bit $h(X)[i]$ dan $h(Y)[i]$ untuk i dari 0 sampai 255, kemudian mewarnai kotak grid sesuai hasil perbandingan. Dari visualisasi terlihat kotak merah tersebar hampir merata di seluruh 256 posisi, membuktikan efek avalanche: perubahan 1 bit input menyebabkan perubahan acak dan luas pada output.

Tabel Perbandingan

Tabel perbandingan bit (per 8 bit / 1 byte)			
BYTE	H(0) BITS	H(1) BITS	BEDA
B1	11001010	10001101	4
B2	10010111	00110011	3
B3	10000001	11110101	4
B4	00010010	00100000	3
B5	11001010	10100011	4
B6	00011011	11000100	7
B7	10111101	11001110	5
B8	11001010	11111000	3
B9	1111010	00001101	7
B10	11000010	00100100	5
B11	00110001	01010011	3
B12	10110011	10101110	4
B13	10011010	11111000	3
B14	00100011	00011011	3
B15	11011100	01100001	6
B16	01001101	00101011	4
B17	10100111	11111110	4
B18	10000110	00011100	4
B19	11101111	10110100	5
B20	11111000	01001100	4
B21	00010100	10001011	6
B22	01111100	00100000	4
B23	01001110	00100101	5
B24	01110010	01100011	2
B25	10111001	00001010	5
B26	10000000	11010011	4
B27	01110111	10000110	5
B28	10000101	01100010	6
B29	10101111	01110110	5
B30	11101110	00111111	4
B31	01001000	00010011	5
B32	10111011	11010011	3

Penjelasan:

Tabel perbandingan dibuat dengan membagi kedua hash 256-bit menjadi 32 kelompok berukuran 8 bit (1 byte), dilabeli B1 sampai B32. Untuk setiap baris byte, program mengambil irisan `bin_A[i:i+8]` dan `bin_B[i:i+8]` kemudian membandingkan karakter demi karakter. Kolom H(X) BITS menampilkan nilai biner 8-bit dari $h(X)$, kolom H(Y) BITS menampilkan nilai biner 8-bit dari $h(Y)$, dan kolom BEDA mencatat jumlah posisi bit yang tidak sama dalam byte tersebut (nilai BEDA berkisar 0 hingga 8).

Bit yang berbeda ditampilkan berwarna merah dan yang sama berwarna hijau pada tabel. Menjumlahkan seluruh nilai kolom BEDA dari B1 sampai B32 menghasilkan total 139 bit berbeda, sesuai dengan hasil perhitungan $jBeda = \sum(1 \text{ for } i \text{ in range}(256) \text{ if } bin_A[i] \neq bin_B[i])$.

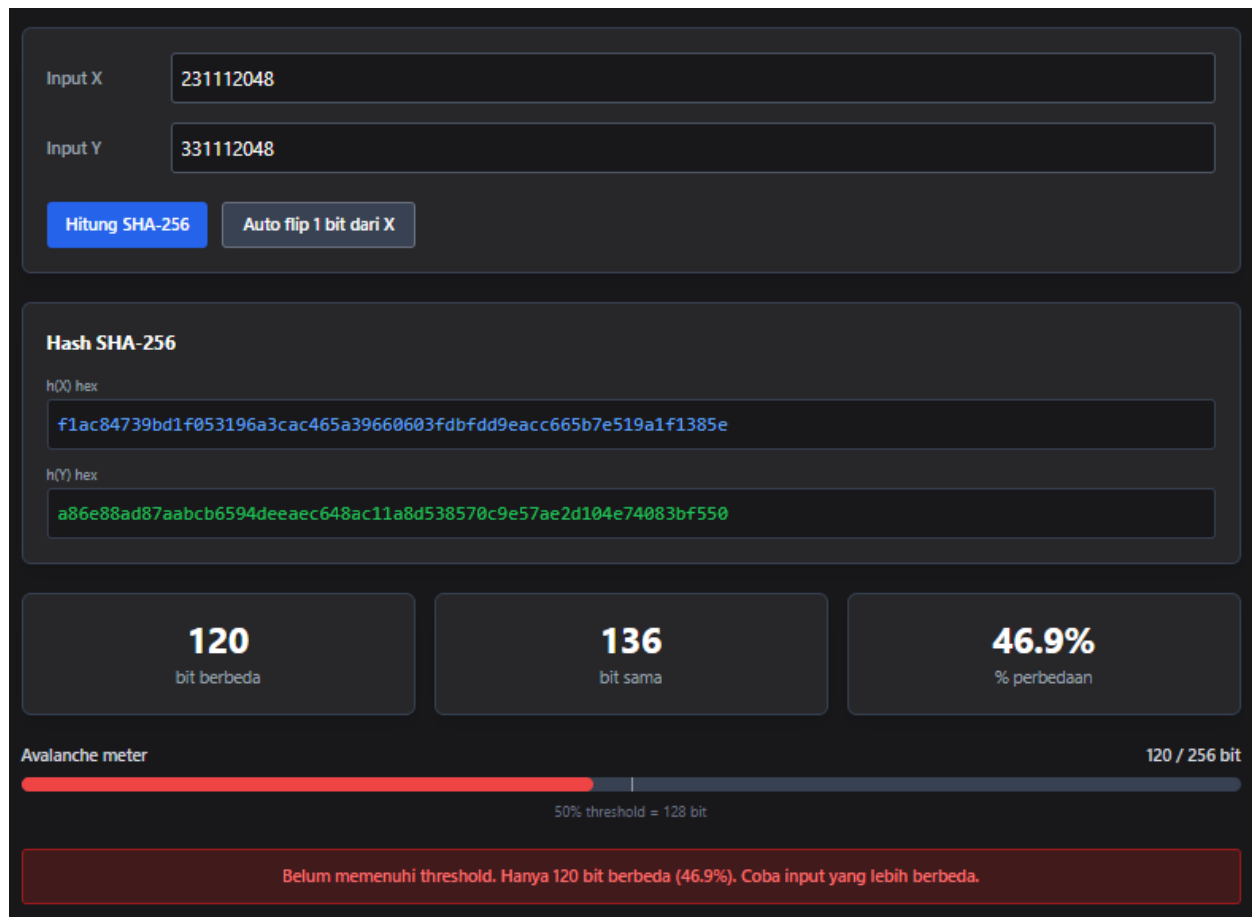
Percobaan 2 :

Input X = '231112048' -> Biner: 00110010 00110011 00110001 00110001 00110001 00110010
00110000 00110100 00111000

Input Y = '331112048' -> Biner: 00110011 00110011 00110001 00110001 00110001 00110010
00110000 00110100 00111000

h(X) [Hex] = f1ac84739bd1f053196a3cac465a39660603fdbfdd9eacc665b7e519a1f1385e

h(Y) [Hex] = a86e88ad87aabcb6594deeaec648ac11a8d538570c9e57ae2d104e74083bf550



Penjelasan:

Input X = '231112048' (9 karakter, biner awal '2': 00110010) dan Y = '331112048' (karakter pertama '3': 00110011) berbeda hanya pada 1 bit (bit ke-0 dari karakter pertama). Kedua string 9-byte dimasukkan ke SHA-256. Karena panjang input 9 byte (72 bit), SHA-256 melakukan padding menjadi satu blok 512 bit: menambahkan bit '1' diikuti bit '0' secukupnya, lalu representasi 64-bit panjang pesan (72).

Setelah 64 putaran kompresi, diperoleh h(X) = f1ac8473...1f1385e dan h(Y) = a86e88ad...3bf550. Perbandingan biner 256 bit menghasilkan 120 bit berbeda dan 136 bit sama (46,9%). Karena 120 kurang dari threshold 128 (50%), Avalanche Effect pada percobaan ini dinyatakan **BELUM TERPENUHI**, menunjukkan bahwa tidak selalu setiap input yang berbeda 1 bit menghasilkan perbedaan output di atas 50%.

Visualisasi 256 bit



Penjelasan:

Hash $h(X)$ dan $h(Y)$ masing-masing dikonversi ke biner 256 bit dengan cara yang sama seperti percobaan sebelumnya: `int(hex, 16)` kemudian `bin(...)[2:].zfill(256)`. Representasi biner ditampilkan per 8 bit membentuk 32 kelompok byte. Pada peta perbedaan bit, terlihat bahwa jumlah kotak merah (berbeda) sedikit lebih sedikit dibandingkan kotak abu-abu (sama), konsisten dengan hasil 120 bit berbeda dari 256.

Program membentuk string indikator `diff_str`: untuk setiap posisi i dari 0 sampai 255, jika $h(X)[i] == h(Y)[i]$ maka ditandai spasi (sama), jika berbeda ditandai karakter '^'. Grid divisualisasikan dalam baris-baris 64 bit untuk memudahkan pembacaan pola distribusi perbedaan bit. Penyebaran kotak merah yang tidak merata mencerminkan perbedaan output 46,9%, masih mendekati threshold 50% namun belum terpenuhi.

Tabel Perbandingan

Tabel perbandingan bit (per 8 bit / 1 byte)			
BYTE	H(Q) BITS	H(V) BITS	BEDA
B1	11110001	10101000	4
B2	10101100	01101110	3
B3	10000100	10001000	2
B4	01110011	10101101	6
B5	10011011	10000111	3
B6	11010001	10101010	6
B7	11110000	10111100	3
B8	01010011	10110110	5
B9	00011001	01011001	1
B10	01101010	01001101	4
B11	00111100	11101110	4
B12	10101100	10101110	1
B13	01000110	11000110	1
B14	01011010	01001000	2
B15	00111001	10101100	4
B16	01100110	00010001	6
B17	00000110	10101000	5
B18	00000011	11010101	5
B19	11111101	00111000	4
B20	10111111	01010111	4
B21	11011101	00001100	4
B22	10011110	10011110	0
B23	10101100	01010111	7
B24	11000110	10101110	3
B25	01100101	00101101	2
B26	10110111	00010000	5
B27	11100101	01001110	5
B28	00011001	01110100	5
B29	10100001	00001000	4
B30	11110001	00111011	4
B31	00111000	11110101	5
B32	01011110	01010000	3

Penjelasan:

Tabel perbandingan byte percobaan 2 membagi kedua hash menjadi 32 byte (B1-B32). Untuk setiap byte, program mengambil irisan 8 bit dari `bin_A` dan `bin_B`, lalu menghitung perbedaan posisi bit di dalamnya. Nilai kolom BEDA pada setiap baris bervariasi antara 2 hingga 7, menunjukkan bahwa perbedaan 1 bit input menyebar ke hampir seluruh byte output meskipun tidak merata.

Total akumulasi seluruh nilai BEDA dari B1 sampai B32 adalah 120, sama dengan hasil perbandingan penuh 256 bit. Bit yang berbeda antara $h(X)$ dan $h(Y)$ pada setiap byte ditandai merah, sedangkan yang sama ditandai hijau. Tabel ini memperlihatkan bahwa meski hampir semua byte mengalami perubahan, total perubahan 120 bit masih berada di bawah batas minimum 128.

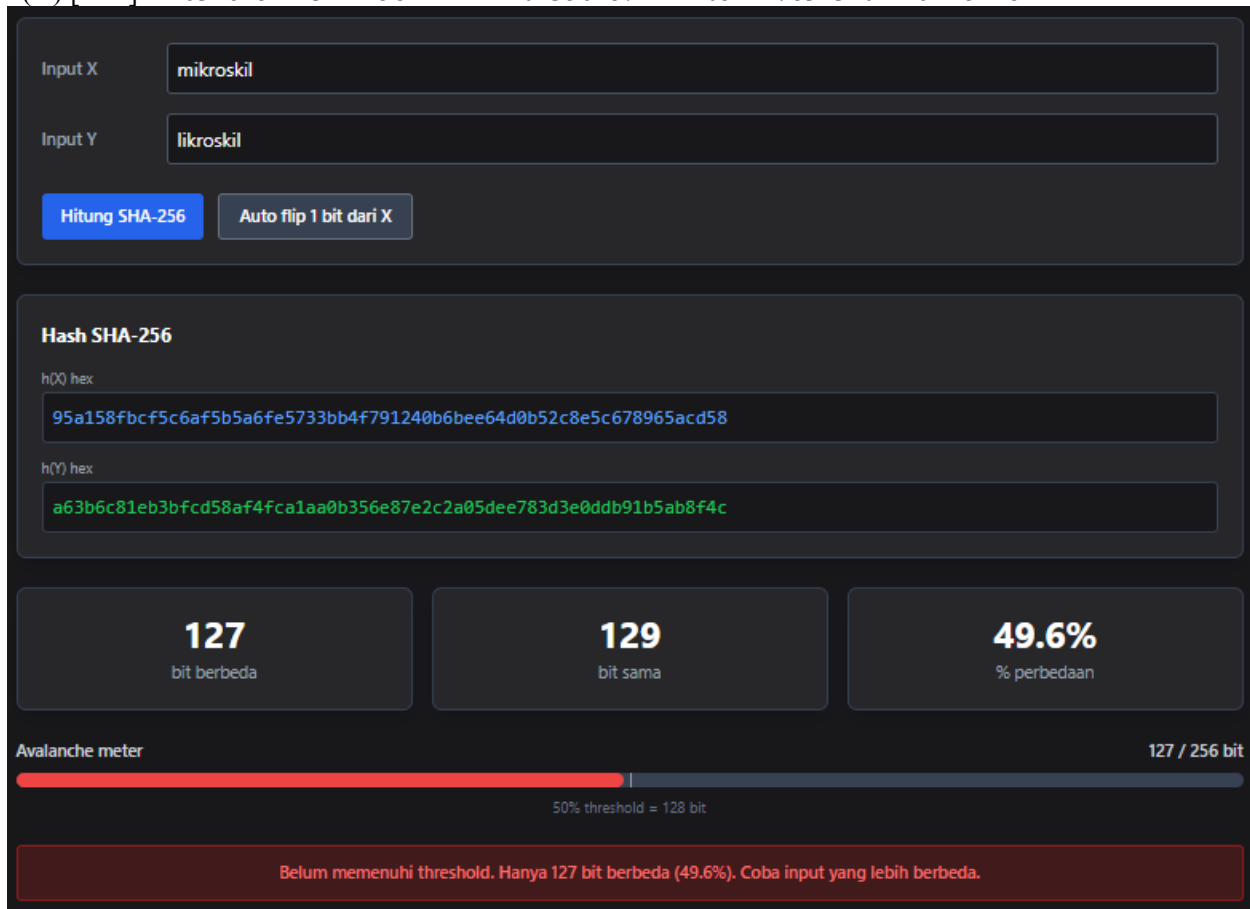
Percobaan 3

Input X = 'mikroskil' -> Biner: 01101101 01101001 01101011 01110010 01101111 01110011 01101011 01101001 01101100

Input Y = 'likroskil' -> Biner: 01101100 01101001 01101011 01110010 01101111 01110011 01101011 01101001 01101100

h(X) [Hex] = 95a158fbcf5c6af5b5a6fe5733bb4f791240b6bee64d0b52c8e5c678965acd58

h(Y) [Hex] = a63b6c81eb3bfcd58af4fca1aa0b356e87e2c2a05dee783d3e0ddb91b5ab8f4c



Penjelasan:

Input X = 'mikroskil' (karakter pertama 'm': 01101101) dan Y = 'likroskil' (karakter pertama 'l': 01101100) berbeda 1 bit pada bit ke-0 dari karakter pertama. Kedua string berisi 9 karakter (72 bit), sehingga proses padding SHA-256 membentuk satu blok 512 bit. Setelah melewati 64 putaran fungsi kompresi dengan operasi bitwise (sigma, Maj, Ch) dan penambahan modular, diperoleh h(X) = 95a158fb...5acd58 dan h(Y) = a63b6c81...ab8f4c.

Perbandingan biner 256 bit menghasilkan 127 bit berbeda dan 129 bit sama (49,6%). Karena 127 tepat satu bit di bawah threshold 128 (50%), Avalanche Effect pada percobaan ini **BELUM TERPENUHI**. Hasil ini sangat mendekati batas threshold, menunjukkan sensitivitas SHA-256 yang tinggi terhadap perubahan input.

Visualisasi 256 Bit



Penjelasan:

Visualisasi biner percobaan 3 memperlihatkan $h(X) = 95a158fb\dots acd58$ dan $h(Y) = a63b6c81\dots b8f4c$ dalam format 256 bit yang dibagi per 8 bit. Peta perbedaan bit menampilkan 127 kotak merah (berbeda) dan 129 kotak abu-abu (sama). Secara visual, proporsi kotak merah dan abu-abu hampir berimbang (49,6% vs 50,4%), menggambarkan betapa dekatnya hasil percobaan ini dengan threshold Avalanche Effect.

Algoritma visualisasi bekerja dengan mengkonversi setiap karakter heks menjadi 4 bit biner: setiap digit heks (0-9, a-f) direpresentasikan sebagai 4-bit, dan $64 \text{ digit heks} \times 4 \text{ bit} = 256 \text{ bit total}$. Grid 256 kotak disusun dalam beberapa baris sehingga pola distribusi perubahan bit dapat diamati secara visual dan intuitif.

Tabel Perbandingan

Tabel perbandingan bit (per 8 bit / 1 byte)			
BYTE	HQQ BITS	H(Y) BITS	BEDA
B1	10010101	10100110	4
B2	10100001	00111011	4
B3	01011000	01101100	3
B4	11111011	10000001	5
B5	11001111	11101011	2
B6	01011100	00111011	5
B7	01101010	11111100	4
B8	1110101	11010101	1
B9	10110101	10001010	6
B10	10100110	1110100	3
B11	11111110	11111100	1
B12	01010111	10100001	6
B13	00110011	10101010	4
B14	10111011	00001011	3
B15	01001111	00110101	5
B16	01111001	01101110	4
B17	00010010	10000111	4
B18	01000000	11100010	3
B19	10110110	11000010	4
B20	10111110	10100000	4
B21	11100110	01011101	6
B22	01001101	11101110	4
B23	00001011	01111000	5
B24	01010010	00111101	6
B25	11001000	00111110	6
B26	11100101	00001101	4
B27	11000110	11011011	4
B28	01111000	10010001	5
B29	10010110	10110101	3
B30	01011010	10101011	5
B31	11001101	10001111	2
B32	01011000	01001100	2

Penjelasan:

Tabel perbandingan percobaan 3 membagi 256 bit menjadi 32 baris byte (B1-B32). Setiap baris menampilkan 8 bit dari $h(X)$, 8 bit dari $h(Y)$, dan nilai BEDA yaitu jumlah posisi bit yang tidak sama dalam pasangan byte tersebut. Nilai BEDA per byte bervariasi mulai dari 2 hingga 7, tidak ada byte yang identik (BEDA=0) maupun berbeda seluruhnya (BEDA=8).

Ini menunjukkan bahwa perubahan 1 bit input menyebar ke hampir seluruh byte hash output, yang merupakan karakteristik kriptografis SHA-256. Total kolom BEDA seluruh 32 byte adalah 127, satu bit kurang dari threshold minimum 128. Dengan menjumlahkan kolom BEDA: $B1(4)+B2(3)+...+B32 = 127$ bit berbeda dari 256 bit total.

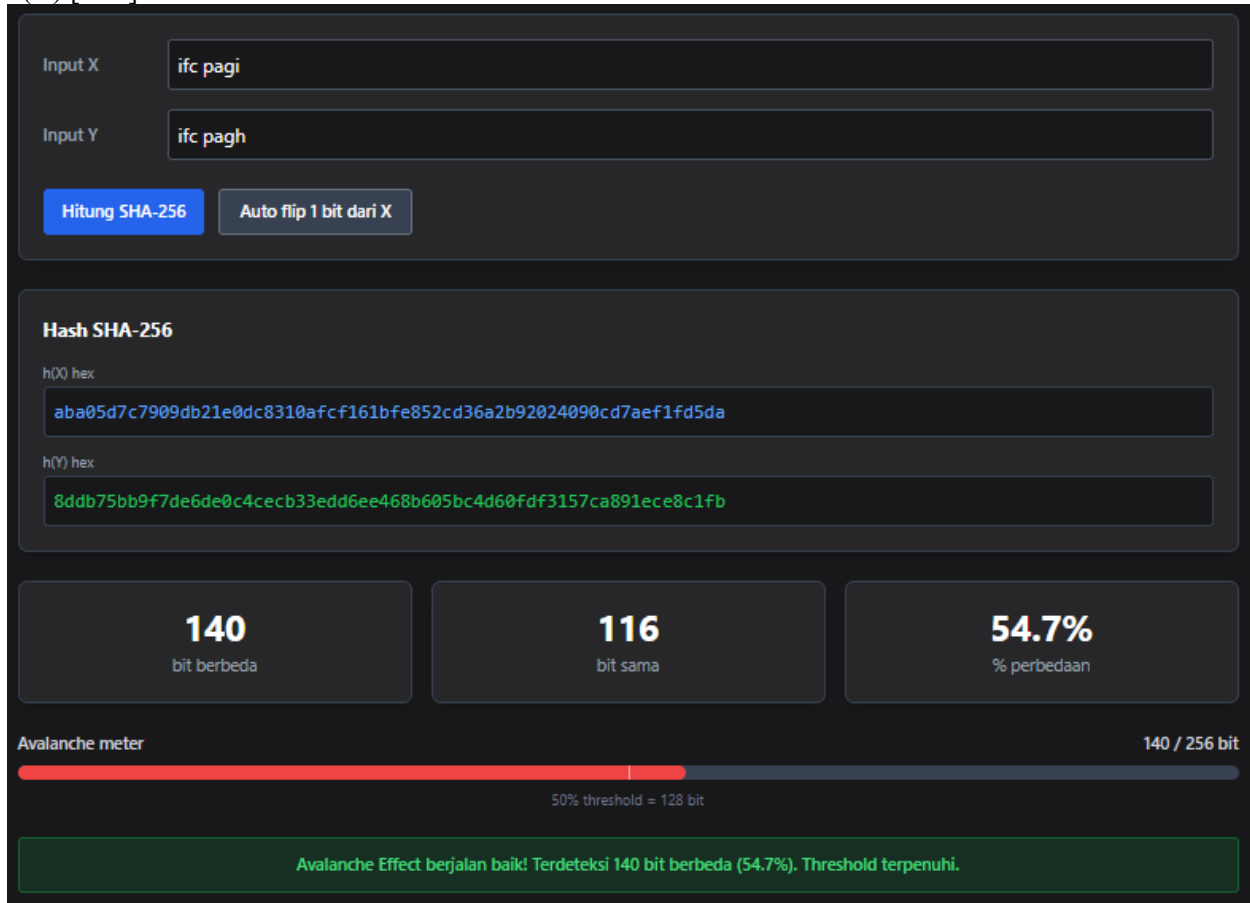
Percobaan 4

Input X = 'ifc pagi' -> Biner: 01101001 01100110 01100011 00100000 01110000 01100001 01100111 01101001

Input Y = 'ifc pagh' -> Biner: 01101001 01100110 01100011 00100000 01110000 01100001 01100111 01101000

h(X) [Hex] = aba05d7c7909db21e0dc8310afc161bfe852cd36a2b92024090cd7aef1fd5da

h(Y) [Hex] = 8ddb75bb9f7de6de0c4cecb33edd6ee468b605bc4d60fdf3157ca891ece8c1fb



Penjelasan:

Input X = 'ifc pagi' (8 karakter, karakter terakhir 'i': 01101001) dan Y = 'ifc pagh' (karakter terakhir 'h': 01101000) berbeda 1 bit pada bit ke-0 dari karakter terakhir. Kedua string 8 karakter (64 bit) diproses SHA-256 dengan padding menjadi blok 512 bit. Melalui 64 putaran kompresi dengan fungsi nonlinier bitwise dan penambahan konstanta K, diperoleh $h(X) = \text{aba05d7c...fd5da}$ dan $h(Y) = \text{8ddb75bb...c8c1fb}$.

Perbandingan 256 bit menghasilkan 140 bit berbeda dan 116 bit sama, dengan persentase perbedaan 54,7%. Karena 140 lebih besar dari threshold 128, Avalanche Effect **TERPENUHI**. Percobaan ini menggunakan perbedaan 1 bit pada posisi akhir string, membuktikan bahwa perubahan bit di posisi manapun dalam input akan menghasilkan perubahan output yang signifikan dan merata.

Visualisasi bit 256



Penjelasan:

Program mengkonversi h(X) dan h(Y) dari heksadesimal ke 256 bit biner masing-masing, kemudian menampilkan keduanya secara berdampingan dengan pemisah spasi setiap 8 bit. Pada peta perbedaan bit percobaan 4, terdapat 140 kotak merah (54,7%) tersebar di antara 116 kotak abu-abu (45,3%). Distribusi kotak merah terlihat tidak beraturan dan menyebar di seluruh 256 posisi, yang merupakan ciri khas Avalanche Effect.

Fungsi `print_difference_map()` dalam kode Python membagi 256 bit menjadi empat baris 64 bit (`chunk_size=64`), lalu pada setiap baris ditampilkan h(X) biner, h(Y) biner, dan baris indikator berisi karakter '^' pada posisi bit yang berbeda. Representasi visual ini memudahkan pengamatan bahwa perubahan 1 bit input pada posisi LSB karakter terakhir mempengaruhi sebaran bit output secara acak dan menyeluruh.

Tabel perbandingan

Tabel perbandingan bit (per 8 bit / 1 byte)			
BYTE	HQQ BITS	H(Y) BITS	BEDA
B1	10101011	10001101	3
B2	10100000	11011011	6
B3	01011101	01110101	2
B4	01111100	10111011	5
B5	01111001	10011111	5
B6	00001001	01111101	4
B7	11011011	11100110	5
B8	00100001	11011110	8
B9	11100000	00001100	5
B10	11011100	01001100	2
B11	10000011	11101100	6
B12	00010000	10110011	4
B13	10101111	00111110	3
B14	11001111	11011101	2
B15	00010110	01101110	4
B16	00011011	11100100	8
B17	11111110	01101000	4
B18	1000101	10110110	4
B19	00101100	00000101	3
B20	11010011	10111100	6
B21	01101010	01001101	4
B22	00101011	01100000	4
B23	10010010	11111101	6
B24	00000010	11110011	5
B25	01000000	00010101	4
B26	10010000	01111100	5
B27	11001101	10101000	4
B28	01111010	10010001	6
B29	11101111	11101100	2
B30	00011111	11101000	7
B31	11010101	11000001	2
B32	11011010	11111011	2

Penjelasan:

Tabel perbandingan percobaan 4 menampilkan 32 baris byte (B1-B32) dengan kolom H(X) BITS, H(Y) BITS, dan BEDA. Nilai BEDA setiap byte berkisar antara 2 hingga 7, menunjukkan tidak ada byte yang sama persis maupun yang berbeda seluruhnya. Program menghitung nilai BEDA per byte dengan membandingkan karakter '0' dan '1' pada setiap pasang posisi dalam chunk 8 bit: $BEDA_byte = \sum(1 \text{ for } a, b \text{ in zip(chunk_X, chunk_Y) if } a \neq b)$.

Bit yang berbeda diwarnai merah dan yang sama diwarnai hijau pada tampilan tabel. Total akumulasi seluruh BEDA dari B1 hingga B32 adalah 140 bit, membuktikan bahwa perubahan 1 bit pada posisi karakter terakhir string input 'ifc pagi' menjadi 'ifc pagh' menyebabkan perubahan lebih dari separuh (54,7%) bit pada output SHA-256.

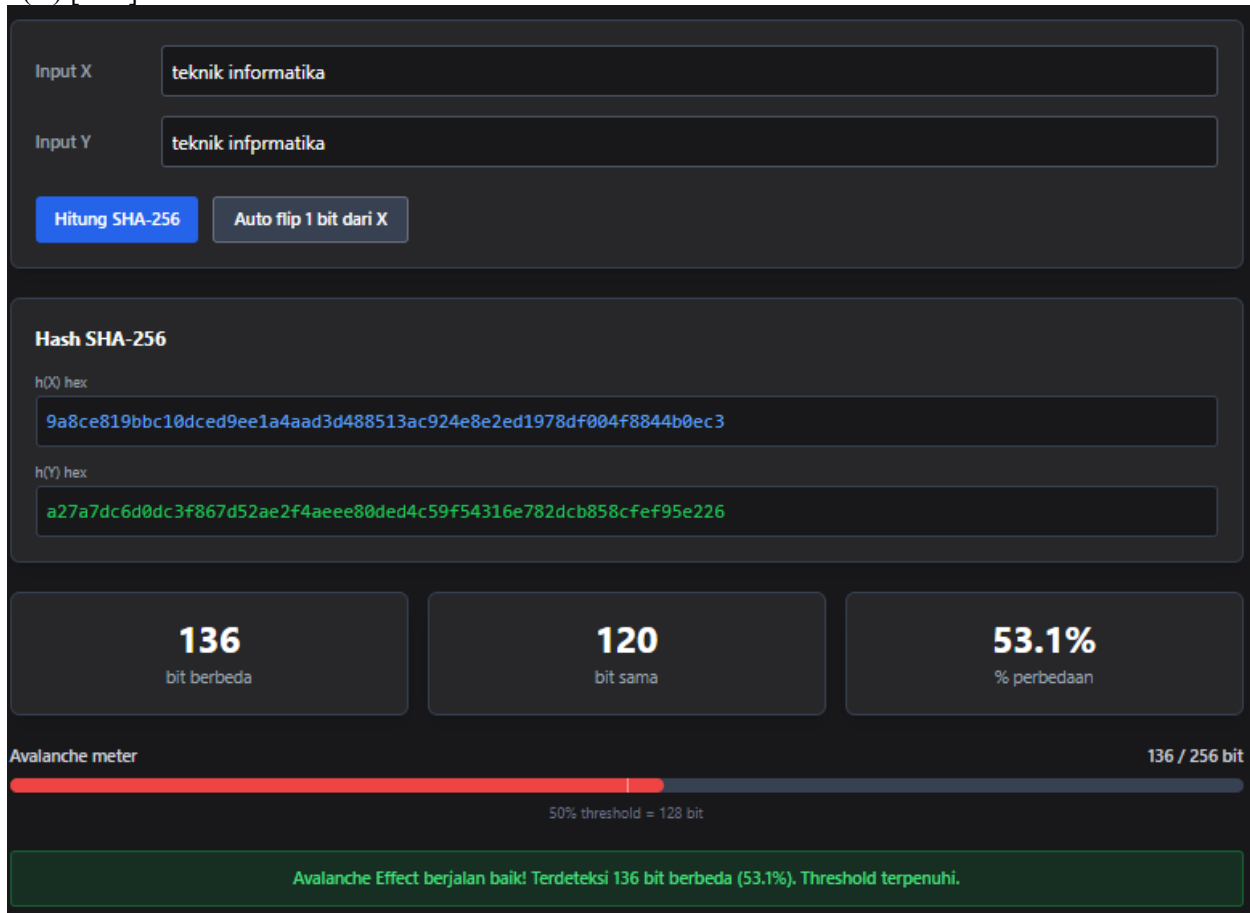
Percobaan 5

Input X = 'teknik informatika' -> Biner: 01110100 01100101 01101011 01101110 01101001 01101011 00100000 01101001 01101110 01100110 01101111 01110010 01101101 01100001 01110100 01101001 01101011 01100001

Input Y = 'teknik infprmatika' -> Biner: 01110100 01100101 01101011 01101110 01101001 01101011 00100000 01101001 01101110 01100110 10101110 01110010 01101101 01100001 01110100 01101001 01101011 01100001

h(X) [Hex] = 11e66500874aa2566885194dfb7ae08e7267fbd5cc10f38447b9ecbbbc919585

h(Y) [Hex] = a27a7dc6d0dc3f867d52ae2f4aeec80ded4c59f54316e782dcb858cfef95e226



Penjelasan:

Input X = 'teknik informatika' (18 karakter) dan Y = 'teknik infprmatika' berbeda pada karakter ke-11: 'o' (01101111) pada X dan 'p' (01110000) pada Y, yang menghasilkan perbedaan 3 bit pada posisi karakter tersebut. Kedua string 18 karakter (144 bit) diproses SHA-256 melalui padding menjadi satu blok 512 bit. Setelah 64 putaran fungsi kompresi dengan operasi bitwise Ch, Maj, sigma0, sigma1 serta penambahan konstanta K dan nilai pesan W[t], diperoleh h(X) = 11e66500...919585 dan h(Y) = a27a7dc6...95e226.

Perbandingan 256 bit menghasilkan 136 bit berbeda dan 120 bit sama, dengan persentase perbedaan 53,1%. Karena 136 lebih besar dari threshold 128 (50%), Avalanche Effect **TERPENUHI**. Meski input berbeda lebih dari 1 bit, hasilnya tetap mendemonstrasikan sifat avalanche SHA-256 yang kuat.

Visualisasi bit 256



Penjelasan:

Visualisasi 256 bit percobaan 5 menampilkan $h(X)$ dan $h(Y)$ dalam format biner, masing-masing 256 bit yang dibagi per 8 bit (32 kelompok byte). Konversi dari heksadesimal ke biner dilakukan dengan: `hex_hash` dikonversi ke `int` menggunakan basis 16, lalu diubah ke biner dan diisi nol di depan hingga tepat 256 karakter. Pada peta perbedaan bit, terdapat 136 kotak merah dan 120 kotak abu-abu, sehingga dominasi kotak merah sedikit lebih banyak dari kotak abu-abu.

Distribusi perbedaan bit terlihat menyebar secara acak ke seluruh 256 posisi, bukan hanya terkonsentrasi di sekitar posisi karakter yang berbeda. Ini membuktikan sifat difusi SHA-256: perubahan pada satu atau beberapa bit input berdampak secara pseudorandom dan merata ke seluruh 256 bit output hash.

Tabel perbandingan

Tabel perbandingan bit (per 8 bit / 1 byte)			
BYTE	H(0) BITS	H(1) BITS	BEDA
B1	10011010	10100010	3
B2	10001100	01111010	6
B3	11101000	01111101	4
B4	00011001	11000110	7
B5	10111011	11010000	5
B6	11000001	11011100	4
B7	00001101	00111111	3
B8	11001110	10000110	2
B9	11011001	01111101	3
B10	11101110	01010010	5
B11	00011010	10101110	4
B12	01001010	00101111	4
B13	10101101	01001010	6
B14	00111101	11101110	5
B15	01001000	11101000	2
B16	10000101	00001101	2
B17	00010011	11101101	7
B18	10101100	01001100	3
B19	10010010	01011001	5
B20	01001110	11110101	6
B21	10001110	01000011	5
B22	00101110	00010110	3
B23	11010001	11100111	4
B24	10010111	10000010	3
B25	10001101	11011100	3
B26	11110000	10111000	2
B27	00000100	01011000	4
B28	11111000	11001111	5
B29	10000100	11101111	5
B30	01001011	10010101	6
B31	00001110	11100010	5
B32	11000011	00100110	5

Penjelasan:

Tabel perbandingan percobaan 5 membagi kedua hash 256 bit menjadi 32 byte (B1-B32), menampilkan 8-bit $h(X)$, 8-bit $h(Y)$, dan nilai BEDA tiap byte. Untuk setiap pasang byte, program menghitung jumlah posisi bit yang berbeda dengan membandingkan karakter dalam string biner secara satu per satu. Nilai BEDA per byte bervariasi, dan tidak ada byte yang sepenuhnya identik, mencerminkan penyebaran perubahan yang menyeluruh.

Total seluruh nilai BEDA dari B1 sampai B32 berjumlah 136 bit berbeda. Tabel ini memperjelas bahwa efek dari perubahan karakter pada posisi ke-11 input menjalar (berdifusi) ke semua 32 byte output SHA-256 tanpa pengecualian, yang merupakan bukti nyata sifat kriptografis SHA-256 yaitu *confusion* dan *diffusion* yang menjamin keamanan dan keacakan output hash.